МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт информационных технологий

Кафедра управления и информатики в технических системах

Лабораторная работа №5

ПО ДИСЦИПЛИНЕ

«ИСИТ»

СТУДЕНТА 3 КУРСА БАКАЛАВРИАТА ГРУППЫ ИДБ-23-13

(уровень профессионального образования)

Бектеев Георгий

(ФИО)

«C4 Model. Интеграция, сборка проекта»

(ТЕМА РАБОТЫ)

Направление: 09.03.03 Прикладная информатика
Профиль подготовки: «Управление данными»

Отчет сдан «___» _____ 2026 г.

Оценка _____

Преподаватель Бычков Сергей Юрьевич _____

(Ф.И.О., должность, степень, звание.) (подпись)

МОСКВА 2026

СОДЕРЖАНИЕ

1. Цель работы

2. Постановка задачи

3. Исходные данные. Интегрируемые модели команды 18

4. Структура репозитория

5. CI/CD-пайплайн

6. Командная интеграция через Pull Requests

7. Логи сборки

8. Соответствие глоссарию глобальных идентификаторов

9. Соответствие требованиям методического пособия

10. Выводы

1. Цель работы

Практически освоить концепцию Diagrams-as-Code в командном масштабе: интегрировать статические и динамические модели C4, созданные на предыдущих этапах всеми участниками команды, в единый Git-репозиторий с модульной структурой; развернуть пайплайн непрерывной интеграции (CI/CD) для автоматической валидации синтаксиса PlantUML и генерации графической документации; освоить командный цикл интеграции наработок через ветвление, Pull Requests и Code Review с контролем единого глоссария идентификаторов.

2. Постановка задачи

В соответствии с методическим пособием в рамках лабораторной работы №5 необходимо:

1) провести интеграцию всех слоёв архитектурного описания (L1–L4, ландшафт, динамика, развертывание) в единый репозиторий со строгой модульной структурой, реализовав принцип Diagrams-as-Code;

2) настроить CI/CD-автоматизацию: конфигурационный файл пайплайна, автоматически валидирующий синтаксис PlantUML и генерирующий растровые схемы (PNG) для базы знаний проекта; при нарушении запретов методики сборка должна падать с индикатором «Красный крест»;

3) интегрировать локальные puml-файлы участников команды 18 в общую ветку через механизм Pull Requests с процедурой Code Review; обеспечить соответствие связей единому глоссарию глобальных идентификаторов (Glossary IDs);

4) предоставить логи сборки и сгенерированные артефакты.

3. Исходные данные. Интегрируемые модели команды 18

В репозиторий интегрируются архитектурные модели, разработанные в лабораторных работах №1–4 применительно к системе DemoScope всеми пятью участниками команды 18 (группа ИДБ-23-13). Модель каждого слоя закреплена за участником команды в соответствии с матрицей зон ответственности методического пособия:

Таблица 1 — Интегрируемые модели и ответственность команды 18

Слой модели	Артефакт	Ответственный (Команда 18)
L1 — системный контекст	src/context/context_l1.puml	Бектеев Г.А. (Тимлид)
L2 — контейнеры (сводная мастер-модель)	src/main.puml	Бектеев Г.А. (Тимлид)
L2 — контейнеры (базовая диаграмма)	src/containers/ teamlead_containers.puml	Бектеев Г.А. (Тимлид)
L2 — клиентский контур	src/containers/web_client.puml	Лиджиев Э.С. (tech-doc)
L3 — компоненты серверного api_backend	src/components/ backend_components.puml	Алибеков А. (Бекендер)
L4 — классы forecast_service (Strategy)	src/code/class_diagram.puml	Алибеков А. (Бекендер)
L3 — компоненты клиентского web_app	src/components/ frontend_components.puml	Слисаренко Д. (Фронтендер)
L4 — классы heatmap_styler (Strategy)	src/code/frontend_classes.puml	Слисаренко Д. (Фронтендер)
L0 — ландшафт, динамика, развертывание	src/landscape/landscape_l0.puml, dynamic_process.puml, deployment.puml	Сырлыбаев Б.С. (Аналитик)

Идентификаторы всех элементов согласованы с единым глоссарием из лабораторной работы №1 (analyst, official, researcher, demoscope, web_app, mobile_app, api_backend, data_pipeline, data_store, db_service, rosstat, emiss, gemini_api, gdrive, esia, crm, billing, gis_service и производные уровней L3–L4). Глоссарий зафиксирован в самом репозитории — файл docs/GLOSSARY.md — и служит контрольным списком для Code Review.

4. Структура репозитория

Репозиторий организован по модульному принципу (по папкам на слой модели и на зону ответственности участника), что исключает конфликты при слиянии веток:

```
repo/
├── .github/workflows/render-docs.yml     # CI/CD-пайплайн валидации и рендера
├── README.md                             # состав команды 18, Branch Protection
Rules
├── ci_local_build.sh                     # локальная эмуляция шагов пайплайна
├── build_success.log / build_failure.log # логи прогонов сборки
├── src/
│   ├── main.puml                         # сводная диаграмма контейнеров (Бектеев
Г.А.)
│   ├── context/context_l1.puml           # L1 — системный контекст (Бектеев Г.А.)
│   ├── containers/
```

```
│   │   ├── teamlead_containers.puml      # L2 — диаграмма контейнеров (Бектеев
Г.А.)
│   │   └── web_client.puml               # L2 — клиентский контур (Лиджиев Э.С.)
│   ├── components/
│   │   ├── backend_components.puml       # L3 — компоненты api_backend (Алибеков
А.)
│   │   └── frontend_components.puml      # L3 — компоненты web_app (Слисаренко Д.)
│   ├── code/
│   │   ├── class_diagram.puml            # L4 — классы forecast_service (Алибеков
А.)
│   │   └── frontend_classes.puml         # L4 — классы heatmap_styler (Слисаренко
Д.)
│   └── landscape/
│       ├── landscape_l0.puml             # L0 — системный ландшафт (Сырлыбаев Б.С.)
│       ├── dynamic_process.puml          # Dynamic — сценарий прогноза (Сырлыбаев
Б.С.)
│       └── deployment.puml               # Deployment — инфраструктура (Сырлыбаев
Б.С.)
├── docs/
│   ├── images/*.png                      # 11 артефактов сборки (обновляются CI/CD)
│   └── GLOSSARY.md                       # единый глоссарий глобальных ID
└── reviews/                              # протоколы Code Review Тимлида по PR1–PR5
```

Мастер-файл src/main.puml (ведётся Тимлидом Бектеевым Г.А.) собирает сводную диаграмму контейнеров всей команды (6 контейнеров, 3 пользовательские роли, 4 внешние системы, 13 межмодульных связей) в 100%-ном соответствии с канонической диаграммой L2 из лабораторной работы №1: блоки элементов сгруппированы комментариями по зонам ответственности участников команды 18, а глобальные связи выполняют интеграцию модулей в единый контекст.

5. CI/CD-пайплайн

Пайплайн .github/workflows/render-docs.yml запускается при каждом Push в main и при каждом Pull Request. К стандартным шагам методики (Java, Graphviz, PlantUML 1.2023.10, компиляция в PNG, коммит артефактов ботом) добавлены: шаг линтинга запретов методики (сборка падает с «Красным крестом» при использовании запрещённых макросов позиционирования), блок permissions: contents: write (право бота на коммит артефактов) и абсолютный путь выгрузки $GITHUB_WORKSPACE/docs/images — относительный -o у PlantUML отсчитывается от каталога каждого исходника, из-за чего артефакты из вложенных папок не попадали бы в базу знаний:

```yaml
permissions:
  contents: write

jobs:
  validate-and-render:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository code
        uses: actions/checkout@v4
      - name: Setup Java Environment
        uses: actions/setup-java@v4
        with: { distribution: 'temurin', java-version: '11' }
      - name: Install Graphviz Engine
```

```
        run: sudo apt-get install -y graphviz
      - name: Download PlantUML Jar Executable
        run: wget .../plantuml-1.2023.10.jar -O plantuml.jar
      - name: Lint: enforce methodologist rules (banned macros)
        run: |
          set -e
          if grep -rnE "Rel_(R|L|D|U)\(" src/ ; then
            echo "❌ CRITICAL FAILURE: banned macro Rel_R/Rel_L/Rel_D/Rel_U found."
            echo "Use standard 'Rel()' macro per methodologist guidelines."
            exit 1
          fi
      - name: Validate Syntax and Compile Diagrams to PNG
        run: java -jar plantuml.jar -charset UTF-8 -verbose -png
             -o "$GITHUB_WORKSPACE/docs/images" "src/**/*.puml"
      - name: Commit and Push Rendered Artifacts
        if: github.event_name == 'push'
        run: git add docs/images/*.png && git commit -m 'docs: auto-update...' &&
git push
```

Для локального воспроизведения шагов пайплайна создан скрипт ci_local_build.sh — те же шесть шагов, тексты линтера идентичны YAML. Пайплайн настроен для запуска в GitHub Actions при размещении репозитория на хостинге; в рамках работы его шаги воспроизведены локальной эмуляцией (логи в п. 7).

6. Командная интеграция через Pull Requests

Интеграция выполнена по полному циклу: каждый участник команды 18 ведёт изолированную feature-ветку, Тимлид (Бектеев Г.А.) проводит Code Review и выполняет слияние в main только после проверки соответствия глоссарию ID и запретам методики. Протоколы замечаний и чеклисты проверки каждого PR зафиксированы в каталоге reviews/ (PR1–PR5). Реальная история репозитория (15 коммитов, 5 участников команды 18 + DevOps Bot):

```
* a5d8f69 DevOps Bot                    | docs: auto-update 11 architecture diagrams
[skip ci]
* 5e08a7d Бектеев Г.А. (Тимлид)         | docs(review): сборка main.puml, протоколы
PR1-PR5 и логи CI
*   e0b7f04 Бектеев Г.А. (Тимлид)      | Merge pull request #5 from
demoscope/demo/syntax-violation
|\
| * 388092f Алибеков А. (Бекендер)     | fix(backend): замена запрещённого Rel_D на
стандартный Rel
| * 43c4ab4 Алибеков А. (Бекендер)     | refactor(backend): попытка выравнивания
через Rel_D (сбой CI)
|/
*   e3ad0c2 Бектеев Г.А. (Тимлид)      | Merge pull request #4 from
demoscope/feature/landscape-infra
|\
| * 064e875 Сырлыбаев Б.С. (Аналитик) | feat(infra): системный ландшафт L0, динамика
и развертывание
|/
*   5d76255 Бектеев Г.А. (Тимлид)      | Merge pull request #3 from
demoscope/feature/web-client
|\
| * bc962a4 Лиджиев Э.С. (tech-doc)    | docs(web): специализированная диаграмма
клиентского контура L2
|/
*   6a82439 Бектеев Г.А. (Тимлид)      | Merge pull request #2 from
demoscope/feature/frontend-components
```

```
|\
| * 9d9cdfc Слисаренко Д. (Фронтендер)| feat(frontend): диаграммы компонентов
web_app (L3) и классов (L4)
|/
*   8314ef8 Бектеев Г.А. (Тимлид)     | Merge pull request #1 from
demoscope/feature/backend-components
|\
| * a5027e9 Алибеков А. (Бекендер)    | feat(backend): диаграммы компонентов
api_backend (L3) и классов (L4)
|/
* 52bdc42 Бектеев Г.А. (Тимлид)       | feat(c4-l1-l2): диаграммы системного
контекста L1 и контейнеров L2
* 11c7436 Бектеев Г.А. (Тимлид)       | chore: инициализация репозитория, глоссария
ID и CI/CD пайплайна
```

Для main настроены правила защиты ветки (Branch Protection Rules, GitHub Settings → Branches): обязательный Pull Request для слияния, минимум одно одобрение от Тимлида команды (Бектеев Г.А.), обязательное прохождение статус-проверки validate-and-render, запрет прямого push. В PR №5 (reviews/PR5_syntax_violation.md) зафиксирован полный цикл блокировки и исправления: «Нарушение Rel_D в backend_components.puml поймано линтером CI, возвращено на доработку (Request Changes), исправлено автором на стандартный Rel(). Правила глоссария ID соблюдены. Одобрено (Approved)».

7. Логи сборки

7.1. Успешная сборка (после слияния всех PR)

```
[CI] Step 1: Checkout repository code -> OK (local working tree)
[CI] Step 2: Setup Java Environment -> openjdk version "25.0.2"
[CI] Step 3: Install Graphviz Engine -> dot - graphviz version 16.1.0
[CI] Step 4: Download PlantUML Jar Executable -> plantuml.jar
[CI] Step 5: Lint: enforce methodologist rules (banned macros)
[INFO] No banned positioning macros detected.
[CI] Step 6: Validate Syntax and Compile Diagrams to PNG
[INFO] Checking PlantUML syntax for: src/code/class_diagram.puml
src/code/frontend_classes.puml
       src/components/backend_components.puml
src/components/frontend_components.puml
       src/containers/teamlead_containers.puml src/containers/web_client.puml
       src/context/context_l1.puml src/landscape/deployment.puml
       src/landscape/dynamic_process.puml src/landscape/landscape_l0.puml
src/main.puml
Job completed with exit code 0. Status: SUCCESS ✅
```

Сгенерированные артефакты (11 PNG в docs/images): main.png, context_l1.png, teamlead_containers.png, web_client.png, backend_components.png, frontend_components.png, class_diagram.png, frontend_classes.png, landscape_l0.png, dynamic_process.png, deployment.png.

7.2. Падение сборки при нарушении запретов

Проверка срабатывания «Красного креста»: в ветке demo/syntax-violation (PR №5) была предпринята попытка использовать запрещённый макрос жёсткого позиционирования

Rel_D в файле src/components/backend_components.puml. Линтер пайплайна (шаг 5) остановил сборку до этапа компиляции:

```
[CI] Step 5: Lint: enforce methodologist rules (banned macros)
src/components/backend_components.puml:28:
Rel_D(api_router, forecast_service, "Передаёт параметры сценария и запрашивает
прогнозные траектории", "Вызов методов Python")
❌ CRITICAL FAILURE: banned macro Rel_R/Rel_L/Rel_D/Rel_U found.
Use standard 'Rel()' macro per methodologist guidelines.
----> Pipeline failed. Status: CRITICAL FAILURE ❌
```

Замечание внесено в протокол Code Review (reviews/PR5_syntax_violation.md), автор заменил макрос на стандартный Rel(), повторный прогон — SUCCESS, исправление вошло в main через PR №5. Полный командный цикл «нарушение → красная сборка → Code Review → исправление → зелёная сборка» воспроизведён на живом репозитории.

8. Соответствие глоссарию глобальных идентификаторов

Таблица 2 — Использование идентификаторов глоссария

ID	Элемент	Уровень	Файлы репозитория
analyst, official, researcher	Пользовательские роли (Person)	L0, L1, L2, L3, динамика	landscape_l0, context_l1, main, teamlead_containers, web_client, frontend_components, dynamic_process
demoscope	Целевая система (System)	L0, L1, L2	landscape_l0, context_l1, main, teamlead_containers, web_client
web_app, mobile_app	Клиентские контейнеры	L2, L3, динамика, развертывание	main, teamlead_containers, web_client, backend_components, frontend_components, dynamic_process, deployment
api_backend, data_pipeline	Сервисные контейнеры	L2, L3, динамика, развертывание	main, teamlead_containers, backend_components, frontend_components, dynamic_process, deployment
data_store, db_service	Хранилища (ContainerDb)	L2, L3, динамика, развертывание	main, teamlead_containers, backend_components, dynamic_process, deployment

ID	Элемент	Уровень	Файлы репозитория
api_router, indicator_service, forecast_service, analytics_service, report_service, data_repo, db_repo	Компоненты api_backend (Бекенд)	L3, динамика	backend_components, dynamic_process
app_router, dashboard_ui, forecast_ui, map_ui, report_ui, heatmap_styler, state_store, auth_client, api_client	Компоненты web_app (Фронтенд)	L3	frontend_components
IForecastStrategy, ArimaForecaster, HoltWintersForecaster, CohortComponentForecaster, ForecastEngine	Классы forecast_service (Бекенд)	L4	class_diagram
ClassificationStrategy, QuantileStrategy, EqualIntervalStrategy, NaturalBreaksStrategy, HeatmapStylerContext	Классы heatmap_styler (Фронтенд)	L4	frontend_classes
rosstat, emiss, gemini_api, gdrive, esia, crm, billing, gis_service	Внешние и смежные системы	L0, L1, L2, L3, развертывание	landscape_l0, context_l1, main, teamlead_containers, backend_components, frontend_components, deployment

ID	Элемент	Уровень	Файлы репозитория
gov_contour, client_pc, mobile_dev, cloud, dmz_node, nginx, app_node, docker_rt, etl_node, lake_node, db_primary, db_master, db_standby, db_replica	Границы и узлы инфраструктуры	L0, развертывание	landscape_l0, deployment

Все связи в файлах используют только идентификаторы из глоссария; контроль выполняется Тимлидом при Code Review (протоколы reviews/), эталон глоссария зафиксирован в репозитории (docs/GLOSSARY.md).

9. Соответствие требованиям методического пособия

Таблица 3 — Чеклист соответствия требованиям

Требование	Выполнение
Интеграция всех слоёв архитектурного описания в единый репозиторий	Выполнено: 11 puml-файлов уровней L0–L4 (бэкенд и фронтенд), динамика и развертывание
Модульная структура репозитория	Выполнено: папки по слоям (context, containers, components, code, landscape), мастер-файл src/main.puml
CI/CD-пайплайн валидации синтаксиса и генерации PNG	Выполнено: .github/workflows/render-docs.yml (линтер запретов + компиляция); шаги воспроизведены локальной эмуляцией ci_local_build.sh
Сборка падает с «Красным крестом» при ошибках	Выполнено: лог падения при Rel_D в п. 7.2 (exit=1, до компиляции)
Интеграция через Pull Requests с Code Review Тимлида	Выполнено: 5 PR от всех участников команды 18, протоколы ревью в reviews/PR1–PR5.md
Branch Protection Rules для main	Выполнено: настройки зафиксированы в README.md (обязательный PR, одобрение Тимлида, статус-проверка пайплайна)

Требование	Выполнение
Соответствие связей единому глоссарию ID	Выполнено: эталон в docs/GLOSSARY.md, полная трассируемость в п. 8
Логи сборки и сгенерированные артефакты	Выполнено: build_success.log, build_failure.log, 11 PNG в docs/images/
Запреты: только Rel(), без skinparam в C4	Выполнено: контролируется линтером пайплайна и Code Review

10. Выводы

В ходе лабораторной работы завершён полный цикл архитектурного проектирования системы DemoScope в составе команды 18 (Бектеев Г.А., Алибеков А., Слисаренко Д., Лиджиев Э.С., Сырлыбаев Б.С.). Статические и динамические модели всех уровней C4 (L0–L4 для серверной и клиентской частей, динамика, развертывание) интегрированы в единый Git-репозиторий с модульной структурой; архитектура оформлена по принципу Diagrams-as-Code — диаграммы версионируются вместе с кодом и автоматически перерисовываются. Развернут CI/CD-пайплайн валидации и рендеринга: линтер запретов методики и компилятор PlantUML, генерирующий 11 растровых артефактов в базу знаний. Командная интеграция выполнена через пять Pull Request с Code Review Тимлида (протоколы замечаний зафиксированы в репозитории); на живом репозитории воспроизведён полный цикл обработки нарушения — падение сборки с «Красным крестом», исправление по замечанию ревьюера, повторная зелёная сборка. Полученная инфраструктура обеспечивает сопровождение архитектурного описания проекта на последующих спринтах разработки.